

Ağ Otomasyonu Veri Formatları

Veri formatları verileri ortak bir yapıda **tanımlamanın**, **depolamanın** ve **değiştirmenin** önemli bir yoludur. Farklı veri formatlarının olmasının sebebi farklı sistemlerin farklı komutlar kullanmasındandır. Ağ otomasyonlarında günümüzde kullanılan **en yaygın veri formatları** şunlardır:

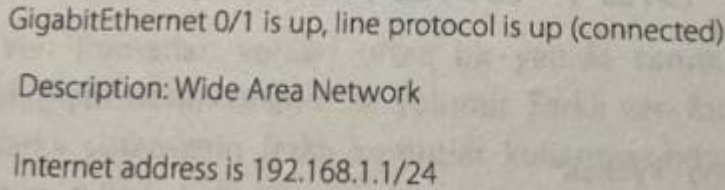
- **XML** (eXtensible Markup language)
- **JSON** (JavaScript Object Notation)
- **YAML** (YAML Ain't Markup Language)

JSON Data Format

JSON web hizmetleri ve API'ler tarafından **herkese açık veriler** sağlamak için kullanılan çok popüler bir veri biçimidir. Çok kullanılmasının en büyük sebepleri **kolay analizi** ve **Python dahil birçok programlama dilinde rahatça kullanılmasıdır**.

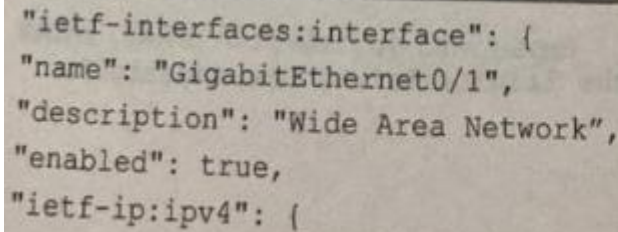
Şimdi, bu veri biçimini daha yakından tanımak için bir **örnek** verelim:

Aşağıdaki bir Router'ın interface'inin durumunu **JSON** formatında ifade edelim. Önce Router'ın interface bilgilerini bakalım:

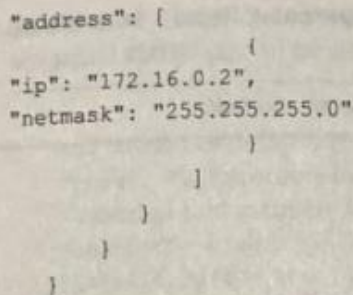


```
GigabitEthernet 0/1 is up, line protocol is up (connected)
Description: Wide Area Network
Internet address is 192.168.1.1/24
```

Şimdi bu veriyi **JSON** dilinde ifade edersek aşağıdaki gibi bir format karşımıza çıkar;



```
"ietf-interfaces:interface": {
  "name": "GigabitEthernet0/1",
  "description": "Wide Area Network",
  "enabled": true,
  "ietf-ip:ipv4": {
```



```
    "address": [
      {
        "ip": "172.16.0.2",
        "netmask": "255.255.255.0"
      }
    ]
  }
}
```

Bu yapıda her değişken ve değerin “ ile **belirlendiğine**, **en son açılan parantezin ilk kapandığında** dikkat ediniz.

JSON Veri Biçiminin Özellikleri ve Kullanım Kuralları

JSON veri formatının bazı belirgin özellikleri aşağıdakilerdir:

- Hiyerarşik, iç içe değerler için idealdir.
- Nesnelerin tanımında {} tırnaklı parantez, dizilerin tanımlarında [] köşeli parantezler kullanılır.
- Veriler key/value çiftleri olarak kullanılırlar.

JSON’da verilerin, key/value çifti olarak kullanıldığını belirtmiştik. Şimdi de bu kullanımda dikkat edilmesi gereken bazı hususlara göz atalım:

- **Keys** değerleri “ ” arasında yer alır.
- Değerler geçerli bir **JSON** veri türünde (dize, sayı, dizi vb.) olmalıdır.
- **Keys/values** kullanımlarında aralarına : konulur.
- Bir nesnenin içinde kullanılacak çok sayıda **key/values** değerleri birbirinden **virgül** ile ayrılır.
- JSON nesnelerinin tanımlanmasında **boşluğun bir zararı yoktur**.

Birden fazla değer içeren **key** değerlerine **array** (dizi) denir. **Array**’lerin kullanımı ile ilgili bazı özellikler şunlardır:

- **Array**’ler sıralı değer listelerinden oluşur.
- **Array**’lerdeki değerler birbirinden **virgül** ile ayrılır.
- Bir **array** sayı, nesne ya da **boole** gibi farklı birçok nesne içerebilir.
- Bir **key** kullanımında :’dan sonra **köşeli parantez** içinde kullanılabilecek değerler girilir.

Öğrendiklerimizi bir örnek üzerinden inceleyecek olursak;

```
{
  "addresses": [
    {
      "ip": "192.168.0.2",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "192.168.0.3",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "192.168.0.4",
      "netmask": "255.255.255.0"
    }
  ]
}
```

Yukarıdaki örneğimizde **IPv4** adreslerinin tanımları yer almaktadır. Buradaki öğrendiğimiz **key** değeri **addresses** kısmıdır. Her bir adres **tırnaklı parantez** ile ayrılmıştır. Her nesne ise **2 adet key/value** çifti içerir. Bunlar **ip** ve **netmask** değerleridir ve dikkat ederseniz nesne içinde kullanımlarında bunlar da birbirinden **virgül** ile ayrılmışlardır. Dikkat ederseniz **addresses** kısmındaki bu diziden hemen sonra : kullanılmış ve tüm tanım **köşeli parantez** içerisinde tanımlanmıştır.

YAML Data Format

YAML da JSON gibi verilerin **okunmasında**, **depolama** ya da **transferinde** kullanılır. YAML kullanımının bazı **özellikleri** şunlardır:

- Tırnaklı, köşeli parantez kullanımı gerekmediğinden daha **sade** bir görünümlü kullanımı vardır.
- Kullanımı JSON'a benzemekle birlikte JSON'ının **bir üst kümesi** olarak kabul edilir.
- Sade yapısından dolayı kolayca **okunabilir**, **yazılabilir**, **analiz edilebilir**.

Şimdi YAML sadeliğini bir örnekle görelim. Bir önceki konuda baktığımız aşağıdaki **JSON** kodunu bir de **YAML** olarak düzenleyelim:

```
{
  "ietf-interfaces:interface": {
    "name": "GigabitEthernet0/1",
    "description": "Wide Area Network",
    "enabled": true,
    "ietf-ip:ipv4": {
      "address": [
        {
          "ip": "192.168.0.2",
          "netmask": "255.255.255.0"
        },
        {
          "ip": "192.168.0.3",
          "netmask": "255.255.255.0"
        },
        {
          "ip": "192.168.0.4",
          "netmask": "255.255.255.0"
        }
      ]
    }
  }
}
```

Bu tanımın YAML kodu:

```
ietf-interfaces:interface:
  name: GigabitEthernet0/1
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
      - ip: 192.168.0.2
        netmask: 255.255.255.0
      - ip: 192.168.0.3
        netmask: 255.255.255.0
      - ip: 192.168.0.4
        netmask: 255.255.255.0
```

Görüldüğü gibi YAML ifade çok daha **sadedir**. Okuması, verileri ayrıştırması, analizi çok daha rahattır. JSON'da bir nesnedeki **key/values** değerlerinin birbirinden ayırmak için **virgül** kullanıyorduk. YAML'da ise **çift tırnak** kullanılmadan sadece **tire** kullanılır.

XML Data Format

XML formatı da JSON ya da YAML gibi verilerin okunması, depolanması ve taşınmasında uygulamalar tarafından kullanılan veri biçimidir. **XML veri biçiminin bazı özellikleri** şunlardır:

- Web sitesi tasarımlarının temeli olan HTML yapısına çok benzer bir kullanıma sahiptir.
- Analizi kolaydır. Her nesne bir **tag yapısı** içinde yer alır. <tag> DATA </tag>
- HTML'den farklı olarak XML'de **önceden tanımlanmış tag** ya da **document yapısı kullanılamaz**.

XML nesneleri de **bir** ya da **birden fazla key/value** değeri içerebilir. Her bir değer tag'ler arasında yer alır ve **açılan bir tag tekrar kapatılmalıdır**.

Örneğin: <key> VALUE </key>

Bu örnekte **VALUE** nesnesi öncesi **<key>** ile tag açılmış, değer tanımlanmış ve sonra **</key>** tag'ıyla kapatılmıştır.

Diğer veri türlerinde incelediğimiz ve aşağıda en sade YAML örneği bulunan interface IP'lerini bir de **XML kodu** olarak tanımlayalım:

Örneğimiz YAML kodu:

```
ietf-interfaces:interface:
  name: GigabitEthernet0/1
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
      - ip: 192.168.0.2
        netmask: 255.255.255.0
      - ip: 192.168.0.3
        netmask: 255.255.255.0
      - ip: 192.168.0.4
        netmask: 255.255.255.0
```

Bu örneğin XML kodu:

```
<?xml version="1.0" encoding="UTF-8" ?>
<ietf-interfaces:interface>
  <name>GigabitEthernet0/1</name>
  <description>Wide Area Network</description>
  <enabled>true</enabled>
  <ietf-ip:ipv4>
    <address>
      <ip>192.168.0.2</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>192.168.0.3</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>192.168.0.4</ip>
      <netmask>255.255.255.0</netmask>
    </address>
  </ietf-ip:ipv4>
</ietf-interfaces:interface>
```

Üstteki örnekte açılan ve kapatılan tag'lere dikkat edin. Her < > arasında kalan ifade bir **tag**'dir ve </tag> ile kapatılmalıdır. JSON ve YAML'da bir nesne içerisindeki değerler **virgül** ya da **tire** değil <tag>'lar ile ayrılır.